

ME 301: Introductory Robotics Laboratory Assignment 3

**By: Tomasz Krzeminski and Jack Press
Date: 03/16/2025**

Abstract

This report investigates the use of a k-Nearest Neighbors (KNN) regression model to address locomotion inaccuracies in a hexapod robot, specifically heading offsets during linear walks. The model utilized ultrasonic sensor scans to predict and correct chassis offsets, achieving an average prediction error of 3.15 degrees and demonstrating effective heading correction during walks with an average error of 0.88 degrees. Despite challenges like electromagnetic interference, the results highlight the potential of machine learning to enhance robotic navigation in dynamic environments.

Introduction

Machines and robots heavily rely on human monitoring and interaction in order to achieve a desired set of goals. This becomes more difficult when the set of goals includes decision-making, adaptability, or autonomy challenges. Machine learning enables these systems to tap into human-like learning schemes which allows them to learn how to achieve tasks, instead of explicitly being programmed to do so. The authors of *Machine Learning (ML): All there is to know* (2024), state that this form of artificial intelligence allows machines to learn these behaviors from data, which is achieved through the optimization of inherent variables, known as model parameters, to improve the predictions which serve as the outputs. The ability to dynamically learn is crucial for tasks that require adaptation to dynamic and uncertain conditions, such as navigation, object manipulation, and human-robot interaction.

As aforementioned, one of the primary motivations for using machine learning within robotics is the challenge of uncertainty. Robots must operate in environments where sensing and actuation are inherently noisy and unpredictable. Traditional control methods, such as control schemes or reactive control, struggle to account for the variability in real-world scenarios, making learning-based approaches an attractive alternative (Mataric, 2007). Machine learning enables robots to process sensory inputs, identify patterns, and make informed decisions, reducing reliance on pre-defined rules and increasing autonomy.

Machine learning methods in robotics can be broadly categorized into supervised learning, unsupervised learning, reinforcement learning, and learning by demonstration. Shinde (2024) states that supervised learning is a type of machine learning which utilizes algorithms to make predictions based on data, which has corresponding features and labels. This approach is commonly used for tasks such as object recognition and classification. Unsupervised learning, on the other hand, allows robots to discover patterns in data without explicit labels, making it useful for clustering and anomaly detection (Mataric, 2007). Reinforcement learning is particularly well-suited for robotics, as it enables robots to learn through trial and error by receiving rewards or penalties based on their actions. This method has been successfully applied to robotic control problems, such as locomotion and robotic arm manipulation (Sutton & Barto, 1998). Learning by demonstration allows robots to learn behaviors by observing human actions, reducing the need for extensive programming (Mataric, 2007).

The implementation of machine learning in robotics requires careful consideration of data collection, algorithm selection, and evaluation metrics. The first step is defining the learning problem, which includes specifying the desired behavior and selecting an appropriate learning algorithm. Data collection is crucial for training machine learning models, and in robotics, this often involves sensor readings, video recordings, or simulated environments. The data is the most important feature in order to develop a robust and reliable machine learning model. Things like data bias, data inaccuracy, incomplete datasets, and non-diversified data can lead to poor accuracy, and therefore poor performance of the models (Kniazieva, 2022). The last step after training the model is performance analysis, which is often conducted on test datasets to determine its effectiveness in real-world applications (Mataric, 2007).

Despite its advantages, machine learning faces several challenges. One major issue is the need for large amounts of training data, which can be time-consuming and expensive to collect. Another challenge is high computational costs due to the large amount of computations necessary in order to process the datasets (GeeksforGeeks, 2023). These large datasets, and input spaces can also lead to increased complexity, and therefore a decreased interpretability, which often makes implementation more difficult. Transfer learning, where a model trained in one domain is adapted for another, has been explored to mitigate these challenges. Furthermore, ensuring safety and robustness in learning-based robotic systems is critical, as errors can lead to unintended consequences (Mataric, 2007).

In order to advance robotics, machine learning will be a fundamental feature in order to enable more sophisticated and autonomous systems. The implementation of learning methods such as, deep learning, probabilistic models, and evolutionary algorithms have enabled these machines to complete more complex tasks.

Future research aims to enhance lifelong learning, where robots continuously learn and improve over their lifetime (Mataric, 2007). Despite the increased capabilities that machine learning provides, it is important to ensure unbiased and ethical data to ensure the highest degree of quality and safety.

Methods

The main challenge that our group faced during the implementation of the behaviors from previous lab assignments was locomotion inaccuracy. The hexapod robot experienced a chassis heading offset, and a linear shift offset whenever it attempted to complete a linear walk. Unfortunately, it is very difficult to pinpoint the exact cause for these offsets, as small things like an unclean floor to more complex things like foot skidding due to imprecise actuation can all have an impact on these inconsistencies. Even with improved locomotion capabilities thanks to an inverse kinematics scheme, the hexapod continued to experience these offsets, albeit to a lesser extent. The heading offset was observed to have the largest impact on the inaccuracy of the robot's walk behavior when compared to the linear shift offset. For this reason, the chosen behavior that the robot would learn is heading correction inside a confined space. The reason for the robot being in a confined space, such as the walls inside a maze, is so that it can determine its position by scanning the walls around it, and associating these scans with the corresponding rotational chassis offset.

In order to teach the robot how to determine its chassis offset, the k-Nearest Neighbors (KNN) algorithm was chosen. According to Christopher (2021), KNN is a supervised learning algorithm which can be used for either regression or classification by predicting the label for an input feature by calculating the distance between the input feature to all the other features in the label-feature space. The algorithm then classifies or performs regression on the labels of the k-nearest neighbors, which means if $k = 3$, then the algorithm would find the shortest distance to the 3 nearest neighbors. Classification is the identification of a specific label within the dataset given the feature, while regression is the prediction of a continuous label by averaging the k-nearest labels (Ramakrishnan, 2024). In order to be able to determine chassis offsets not within the training dataset, a regression KNN model will be used.

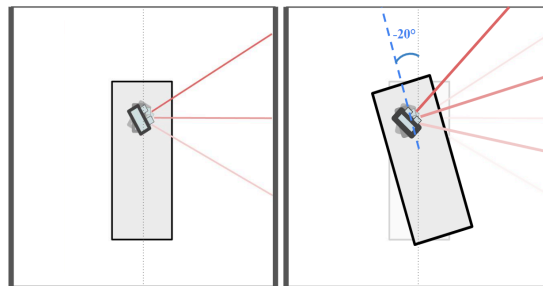


Figure 1: No Chassis Offset Scan (Left) vs. -20° Chassis Offset Scan Left

The label space of our KNN model will be composed of the chassis offset, measured in number of degrees, and the corresponding feature space will be a vector composed of 13 ultrasonic sensor scans. Figure 1 shows the chassis of the hexapod, represented by the large rectangle, and an ultrasonic sensor, which is positioned in three different scan orientations. The 13 scans are interpolated between 0 degrees, and 60 degrees, meaning that the ultrasonic sensor takes a scan every 5 degrees. Figure 1 also shows two different configurations of the chassis, where the left image corresponds to a perfect chassis alignment with the ultrasonic sensor scans represented by the red lines, and the right image, which corresponds to a chassis offset of 20 degrees to the left. From these images, it can be seen that the distance from the wall to the ultrasonic sensor is different at each scan angle given a different chassis offset. This variation in scan distances at different chassis offsets allows for a unique classification of labels. For these reasons, the input to our KNN regression model will be the 13-vector of ultrasonic sensor scans, and the output will be the chassis offset angle, which will be determined by taking the average of the k-nearest neighbor's labels, and used by the robot by turning its chassis by the provided angle.

For this implementation to be successful, a few assumptions had to be made. The first assumption was that the wall placement within the maze was near-perfect, meaning that the only offset corresponding to the robot was its very own chassis offset. The second assumption was that the ultrasonic sensor scan will be able to differentiate between small angle shifts, and therefore provide us with a viable resolution for the input space. The

third assumption was that the robot will not necessarily be in the perfect center of the maze tile (i.e. it experiences linear offset). The final assumption was that the robot will always be in a confined space with either one wall next to it, or one wall next to and one wall in front of it (2 configuration spaces). This meant that if a robot scanned its right side and detected no wall, it would have to perform scans on its left side. If the robot found no walls on both sides, it would not be able to perform a correction. The main unknown of this assignment was whether the robot was able to turn its chassis at the exact commanded angle.

To tune the performance of the algorithm four things can be done. The first tuning method is to vary the k-value, which will determine how many neighbors the input vector will be compared against. The second tuning method is to reduce the size of the input space. The way our group went about this was by removing values from the input and training vectors by removing values at positions with user-defined modulus values (i.e. 13 % 2 would reduce the feature vector from 13 values to 8 values). The third method of tuning the KNN algorithm was by increasing the resolution of the training dataset. The final tuning method was to utilize different distance metrics (i.e. Euclidean Distance or Cosine Distance), to determine the distance between data points.

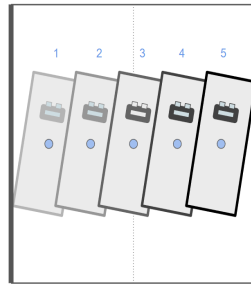


Figure 2: 5 Different Linear Offsets at 10 Degree Chassis Angular Offset

The data necessary for training the model is composed of 90 different data points, which are composed of 45 corresponding to a spatial configuration of only a wall next to the robot, and another 45 corresponding to a spatial configuration of a wall next to and in front of the robot. The 45 data points are composed of 9 different chassis offset angles (-30, -20, -15, -10, 0, 10, 15, 20, 30), and at each chassis offset, there are measurements taken at 5 different linear offsets as seen in Figure 2. To gather this data, the robot will be placed inside the aforementioned spatial configuration of the maze. Inside the maze tile, the robot will first be perfectly centered, and have zero heading offset. Using a compass, the robot's body will be turned by the aforementioned chassis offsets, and then offset linearly in one of the five different linear offset configurations. These linear offset configurations do not need to be measured precisely, and in our procedure we had the closet leg to either wall in the right most and left most configurations touching the wall, the center configuration had the center of the chassis aligned with the center of the maze tile, and the two in between configurations were achieved by placing the center of the chassis between the central and outer configurations. This is the optimal number of data entries so as to not increase computational complexity. To create the dataset, the robot will feed a 14-vector, with the first entry being the chassis offset, and the rest of the values being the sensor readings, into a CSV file.

To determine the performance of the model, the robot will first be placed inside the maze at 19 different chassis offsets, not included in the training data set. These configurations will be set up similar to the training data, however, only the chassis angle will be measured, and the linear offset will be random. This method of testing the correction is important because when the robot experiences both offsets during the walk, its linear offset will always be random. The second procedure for analyzing the performance of the model will be to set up the robot inside the maze, have it walk one tile, and then measure and correct the angular chassis offset. The robot will naturally experience linear and angular offsets as it walks, which allows us to see how well the model corrects dynamic error. In addition to this, it is important to vary the starting configurations. There should be configurations in which the robot is perfectly centered and aligned, and then there should be configurations where the robot starts at an offset configuration, and walks from there. Additionally, there should be tests where the robot reads a wall to its right, and then tests in which it reads no wall to its right, at which point it should gather its readings from the left wall. The tools necessary to perform this analysis are a compass, which will be used to measure the different chassis angles, and a recording device to capture the performance for later analysis.

Results

1	30	255	255	259	259	256	261	283	370	370	366	362	358	291
2	30	274	274	274	278	283	288	298	377	372	372	377	381	276
3	30	306	306	306	306	307	316	334	340	425	404	391	404	275
4	30	339	335	335	336	341	341	347	424	423	421	425	273	272
5	30	363	367	367	368	373	374	379	425	395	425	428	286	277

Figure 3: Training Dataset Excerpt for 30 Degree Chassis Offset

Heading Correction	Starting Chassis Offset (deg)	Predicted Correction Angle (deg)	Prediction Error (deg)	Angle Turned (deg)	Final Chassis Offset (deg)	Scan Direction (Left: 0 Right: 1)	Scan Execution Time (s)
Trial 1	-13	-15.56	-2.56	-16	-3	0	20.81
Trial 2	13	16.11	3.11	16	3	0	20.81
Trial 3	24	24.44	0.44	25	1	0	20.81
Trial 4	-21	-16.11	4.89	-16	5	0	20.80
Trial 5	-18	-18.89	-0.89	-20	-2	0	20.80
Trial 6	9	12.78	3.78	12	3	0	20.80
Trial 7	3	0.00	-3.00	0	-3	0	20.80
Trial 8	-7	-3.33	3.67	-2	5	0	20.80
Trial 9	12	16.67	4.67	17	5	0	20.79
Trial 10	-8	-6.67	1.33	-7	1	0	20.79
Trial 11	-28	-27.78	0.22	-26	2	0	20.79
Trial 12	-13	-15.56	-2.56	-16	-3	0	20.79
Trial 13	18	16.11	-1.89	16	-2	0	20.79
Trial 14	21	24.44	3.44	25	4	0	20.79
Trial 15	-24	-16.11	7.89	-16	8	0	20.78
Trial 16	12	7.22	-4.78	7	-5	1	30.06
Trial 17	-10	-12.78	-2.78	-13	-3	1	30.06
Trial 18	26	20.00	-6.00	20	-6	1	30.06
Trial 19	3	1.11	-1.89	1	-2	1	30.06
Average	-	-	3.15	-	3.47	-	22.75
Standard Deviation	-	-	1.94	-	1.81	-	3.88

Table 1: Heading Correction Data

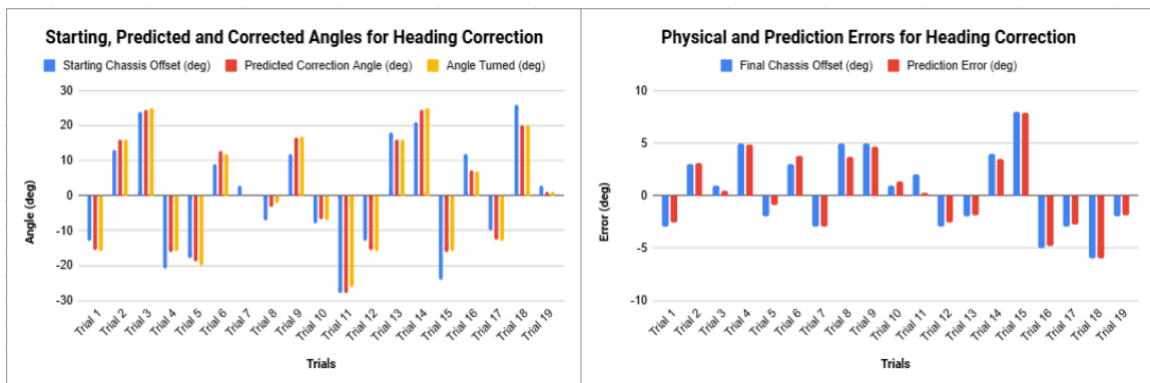


Figure 4: Angle Data (Left) and Error Data (Right) for Heading Correction

The data gathering for the training dataset was performed in 2 different spatial configurations at 9 different angular chassis offsets, with 5 different linear chassis offsets per angular offset. This data collection was performed on a clean floor with the walls of each individual spatial configuration set up as straight as possible. Figure 3 shows an excerpt of 5 entries from the training data set, all corresponding to a chassis offset of 30

degrees. The first value in each vector corresponds to the label, and the remaining 13 values correspond to the feature.

The performance analysis for heading correction was performed in the same conditions as the training data. The chassis angle was measured with a compass, and the scan execution time was measured by an programmed internal counter in the robot. A recording of the whole procedure was taken to determine the initial chassis offset, as well as the angle turned as calculated by the model. The predicted angle output by the model was measured by having the robot write the vector of scanned values, and the predicted angle to a CSV file.

Walk Heading Correction	Chassis Offset at End of Walk (deg)	Predicted Correction Angle (deg)	Prediction Error (deg)	Angle Turned (deg)	Final Chassis Offset (deg)	Scan Direction (Left: 0 Right: 1)	Scan Execution Time (s)
Trial 1	-4	-2.22	1.78	-4	0	0	18.80
Trial 2	2	2.22	0.22	2	0	0	20.81
Trial 3	2	1.11	-0.89	2	0	0	20.81
Trial 4	1	1.11	0.11	2	1	0	20.81
Trial 5	-2	-2.22	-0.22	-1	1	0	20.81
Trial 6	1	2.22	1.22	3	2	0	20.82
Trial 7	2	2.22	0.22	2	0	0	20.82
Trial 8	2	0.00	-2.00	0	-2	0	18.82
Trial 9	-13	-15.00	-2.00	-15	-2	0	20.81
Trial 10	4	3.89	-0.11	4	0	0	20.81
Trial 11	16	13.33	-2.67	13	-3	0	20.81
Trial 12	15	17.22	2.22	15	0	0	20.82
Trial 13	-3	-2.22	0.78	-2	1	1	33.83
Trial 14	-2	-2.22	-0.22	-2	0	0	20.82
Trial 15	0	0.00	0.00	0	0	1	31.84
Trial 16	2	2.22	0.22	2	0	0	20.81
Trial 17	1	1.11	0.11	1	0	1	33.84
Average	-	-	0.88	-	0.71	-	22.76
Standard Deviation	-	-	0.91	-	0.99	-	5.03

Table 2: Walk Heading Correction Data

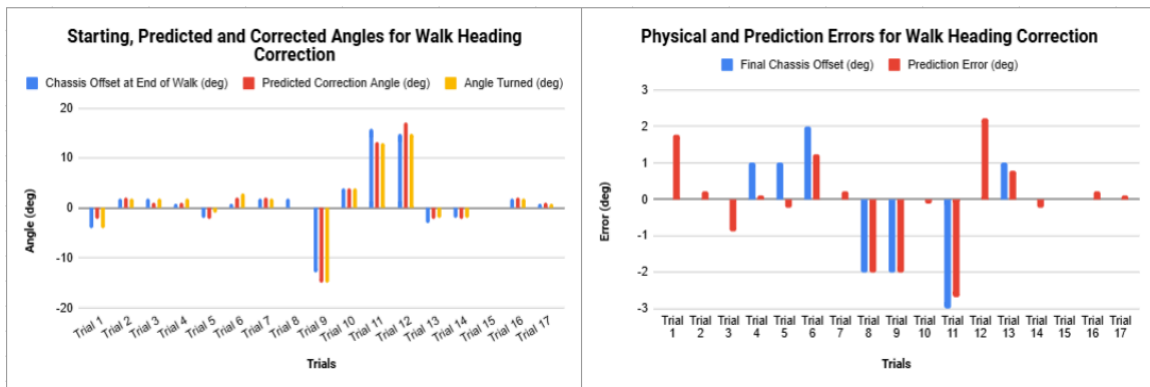


Figure 5: Angle Data (Left) and Error Data (Right) for Walk Heading Correction

The performance analysis for the walk heading correction was performed in the same conditions as the training data, and the heading correction analysis, however, the path was extended to simulate a segment of the maze from the previous lab assignment. Additionally, there was a section in the path which had walls missing, which would allow our group to observe how the robot behaved if it did not detect walls on one of its sides. Furthermore, the starting position and angle of the robot were varied in some of the tests. The chassis angle was measured with a compass, and the scan execution time was measured by an programmed internal counter in the robot. A recording of the whole procedure was taken to determine the chassis offset at the end of each walk, as well as the angle turned as calculated by the model. The predicted angle output by the model was measured by having the robot write the vector of scanned values, and the predicted angle to another CSV file.

Discussion

The heading correction results show that the predictions of the KNN regression model on average yield a prediction error of about 3.15 degrees, with a standard deviation of 1.94 degrees as is seen in Table 1. Furthermore, it can be seen from Figure 4 that the actual turn behavior of the robot closely follows the predicted

turn angle as is evident from the red and orange lines. The error data in Figure 4 also shows that the final chassis error is directly proportional to the prediction error, and is almost identical to it. This result implies that the physical turn behavior of the robot is very accurate, even to small degrees. This can be seen in Table 1, where Trial 19 shows that the predicted angle is 1.11 degrees, and the actual turn is exactly 1 degrees. The angles in Figure 4 also show that despite not perfectly predicting the angle needed to fix the chassis offset, each prediction correctly identifies the sign of each response, and generally misses the goal on average by 3.15 degrees as mentioned above. Another interesting observation from Figure 4 is that despite having a finer resolution between 10 and 20 degrees, due to there being data for chassis offsets of 10, 15 and 20 degrees, the KNN model still had trouble predicting exact response angles. This data suggests that the 15 degree data might not be necessary for good performance.

The walk heading correction results show that the KNN model is very accurate for correction during the walk, as is indicated by the average prediction error of 0.88 and average final chassis error of 0.71 degrees. This is important because on average the robot experienced around 2 degrees of heading error as it concluded each walk, which can be seen in Table 2 for all the trials with low chassis offsets. These walks were started with the robot being configured at a perfect start position. The error data in Figure 5 shows that the robot had very little prediction and chassis offset errors post correction. This results suggests that for an average walk, the correction would suffice, and allow the robot to correct itself enough to not compound error. Additionally, Figure 5 shows that even if the robot experienced relatively large angular offsets of around 15 degrees, it was capable of correcting itself, with an error threshold of around 3 degrees. This also suggests that if the robot performs a double correction, it would theoretically be possible to further correct the error that large corrections experience.

The following results show that the model is capable of predicting the chassis offset angle, with relatively low error. The more important thing to consider is that the robot is capable of predicting and correcting small angle offsets relatively accurately. It is not perfect by all means, however, if the small angle detection resolution were increased, it is possible that the robot could perfectly correct its angular offset. The aforementioned results from Figure 4 showing that despite having a finer resolution, the prediction was no better, implies that it might be more beneficial to have a rougher resolution for larger angle corrections, and then increase the resolution accuracy for smaller angles. This improvement could then be paired with a modified corrective behavior of having the robot imprecisely correct the large angle error, and then precisely correct the small angle error. In this way, the dataset length is able to remain the same size, and possibly have better performance. The only downside is that the correction process would take a longer amount of time.

A large source of uncertainty for this lab is the accuracy of the compass measurements. It was observed during data collection that the compass would start to change its readings as the ultrasonic sensor performed its scans. This most likely occurs due to electromagnetic interference, which impacts the magnetic field around the compass, and causes its readings to change. Because of this, it was impossible to be certain about some of its readings, and it is possible that readings like those seen in Table 2, Trial 1, where the measured chassis offset is 2 degrees larger than the predicted angle. This uncertainty could also mean that the predicted angles are more accurate than may seem, as an uncertainty of 1 degree or more is relatively large when measuring angles ranging from 0 to 30 degrees. Overall, despite the challenges with gathering results, the learned behavior was very successful, and if implemented in a maze exploration, it would greatly improve its traversal abilities.

Conclusion

The implementation of the KNN regression model for heading correction in the hexapod robot demonstrated promising results in addressing locomotion inaccuracies. The model successfully predicted chassis offsets with relatively low error, enabling the robot to correct its heading during walks. The average prediction error of 0.88 degrees and the final chassis error of 0.71 degrees indicate that the robot can effectively navigate confined spaces, such as mazes, with improved accuracy. However, challenges such as electromagnetic interference affecting compass readings and the need for finer resolution in small angle corrections highlight areas for future improvement. Overall, the learned behavior significantly enhanced the robot's ability to navigate dynamic environments, suggesting that machine learning techniques, particularly KNN, can be valuable tools for improving robotic autonomy and adaptability in real-world applications. Future work could focus on refining the model's resolution for small angle corrections and addressing electromagnetic interference to further enhance performance.

References Cited

- Machine learning (ML): All there is to know*. ISO. (2024, January 10).
<https://www.iso.org/artificial-intelligence/machine-learning>
- Matarić, M. J. (2007). *The Robotics Primer*. The MIT Press.
- Shinde, S. (2024, February 26). *What is supervised learning and its top examples?*. Emeritus Online Courses.
<https://emeritus.org/blog/ai-and-ml-supervised-learning/>
- Sutton, R. S., & Barto, A. G. (1998). *Reinforcement learning: An introduction*. MIT Press.
- Kniazieva, Y. (2022, September 15). *Data collection*. High quality data annotation for Machine Learning.
<https://labeleyourdata.com/articles/data-collection-methods-AI>
- GeeksforGeeks. (2023, January 30). *Advantages and disadvantages of Deep Learning*. GeeksforGeeks.
<https://www.geeksforgeeks.org/advantages-and-disadvantages-of-deep-learning/>
- Christopher, A. (2021, February 3). *K-Nearest Neighbor*. Medium.
<https://medium.com/swlh/k-nearest-neighbor-ca2593d7a3c4>
- Ramakrishnan, M. (2024, December 2). *What is classification in Machine Learning and why is it important?*. Emeritus Online Courses.
<https://emeritus.org/blog/artificial-intelligence-and-machine-learning-classification-in-machine-learning/>